FlashANNS: Hiding NAND from Graph Search on Flash-Backed CXL Memory

Anonymous Author(s)

Abstract

Billion-scale graph ANNS needs a flash-priced corpus and a hot path a query host can score from. Block-SSD systems such as DiskANN hide NAND behind asynchronous I/O and PQ navigation, but remain a per-host block serving unit. Full-in-memory CXL-DRAM systems such as CXL-ANNS and Cosmos remove the flash miss entirely and price the cold tail at DRAM. Flash-backed CXL memory is the third point: full-precision vectors remain on Flash, a 4 GiB CXL-DRAM page cache exposes memory-semantic access, and host DRAM keeps the adjacency graph, compact PQ codes, and a smaller scoring window. A unified VA does not make search usable. The runtime question is what must be hidden so that graph walk does not wait on NAND.

We present FlashANNS, a graph-ANNS runtime that splits that contract. *Score hide* permits a host-cache or CXL-DRAM-cache hit, but no scoring access may trigger or wait for a Flash fill. *Query hide*—the timed path also does not wait for a fill—is a stronger contract, not a result we claim. Host-side PQ navigation first commits a bounded rerank set; wise prefetching then fetches only its missing full-precision vector pages, using co-use-aware placement and dense page requests to increase useful bytes per read. Continuous batching overlaps one query's committed-page materialization with navigation or reranking from other queries, sustaining bounded Flash parallelism without changing per-query search semantics. The evaluation separates host-cache hits, CXL-DRAM hits, and Flash fills at iso-recall; compares the three-tier path with demand access and block-SSD search across three datasets; and isolates both design mechanisms. A same-machine PipeANN block path provides a named block-I/O comparison at matched recall. We evaluate a dual-NVMe Flash backing through the software memory-semantic manager `vmem_sw`; we do not equate that manager with a product CXL.mem SSD or claim 160 GB/s, and we treat multi-host one-copy as an unevaluated Shared-FAM deployment model.

CCS Concepts: • **Computer systems organization** → **Secondary storage organization**; • **Information systems** → **Nearest-neighbor search**; • **Software and its engineering** → *Memory management*.

Keywords: CXL, CXL-SSD, approximate nearest neighbor search, graph ANNS, memory-semantic flash, residency, prefetch, continuous batching

1 Introduction

Graph-based approximate nearest neighbor search (ANNS) is a production serving primitive: a query walks a proximity graph and scores neighbors until a beam budget is spent. At billions of vectors the full-precision embeddings no longer fit in host DRAM, even when the more compact graph and PQ metadata do, so the vector payload must live behind another tier. CXL makes that tier addressable with `load/store`. When the whole corpus fits in CXL-DRAM, prior systems already offer a usable service—CXL-ANNS and Cosmos [15, 20] never stall on flash. This paper is about the next device. Flash-backed CXL memory exposes Flash capacity through a bounded CXL-DRAM page cache on a CXL.mem-style address space [6, 22, 41, 42]. A query host may keep a still smaller software cache, but a miss in the CXL-DRAM cache costs tens of microseconds because it fills from Flash. To our knowledge, FlashANNS is the first *graph-ANNS serving runtime* that operates directly over an SSD-backed CXL address space and enforces *score hide*. The host computes from its small cache or from a CXL-DRAM-cache hit; no scoring access may trigger or wait for a Flash fill. We do not claim the first CXL-DRAM ANNS, the first CXL-SSD attached to a vector database, or a product CXL.mem SSD.

Two published lines each solve one side of the capacity–latency tension and tax the other (Figure 1). DiskANN [31, 32] and pipelined descendants such as PipeANN [13] are excellent *block-SSD* ANNS: PQ navigation, sector layout, and asynchronous I/O hide NAND behind a per-host cache. Their serving unit remains CPU + DRAM working set + a local SSD index. Scaling the service across hosts commonly replicates or shards the flash-resident index. They hide *block I/O*, not a memory-semantic hierarchy with two managed cache levels. CXL-ANNS and Cosmos put the graph and embeddings in a CXL-DRAM pool. Search never sees flash, but the cold tail $D - H$ is priced at DRAM [5, 19]. We build a third service because neither line is the right contract for flash-backed CXL memory: full-precision vectors should stay Flash-priced, compact search information should guide movement from host DRAM, and the runtime must hide Flash fills from the scoring instruction. Multi-host one-copy under Shared FAM is a deployment path we do not evaluate (§2).

Making that service real is hard because the device is not slightly slow DRAM and not a DiskANN SSD.

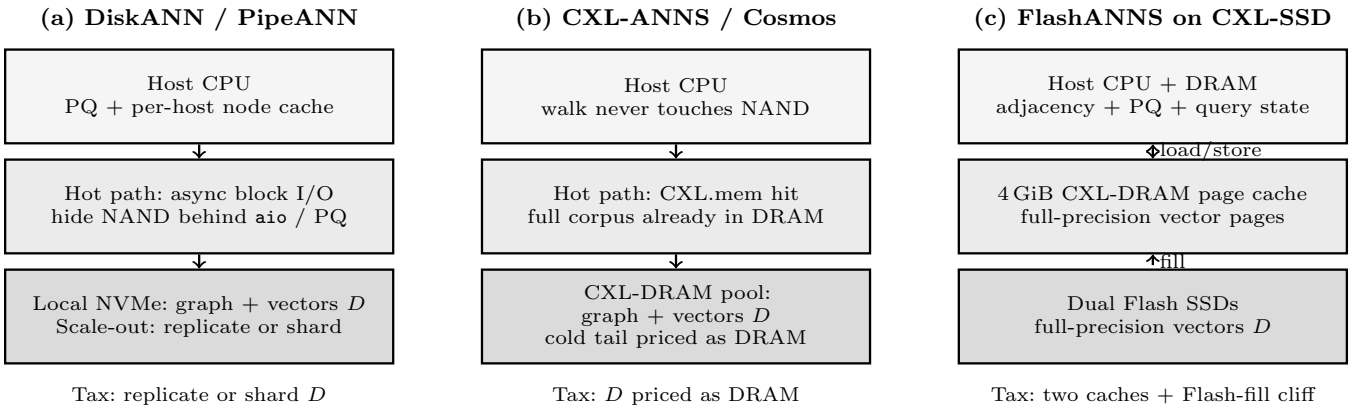


Figure 1. Two published deployments and the third point this paper serves. Flash-backed CXL memory is the *opportunity*: Flash-priced D, a 4 GiB CXL-DRAM page cache, and a small host cache. FlashANNS is the *runtime*: a wise prefetcher plus continuous batching so scoring does not wait on NAND. Host DRAM holds search information; the two lower tiers cache and store the full-precision payload. Shared FAM is not a measured link.

Three hardware–search interactions break conventional designs (§3). *(C1) The miss cliff.* A CXL-DRAM-cache hit is a DRAM-class access; on our measured path, a Flash fill takes about 89 μs, or 130–320× the latency of a CXL-DRAM item load. Graph walk multiplies the tax by hops × degree, so treating the composed hierarchy as uniform mmap'd memory makes search miss-bound. *(C2) Prefetch must be wise.* Neighbor IDs are not sequential. OS-style page admission and synchronous 2-hop prefetch bring records the walk will not use, pollute the 4 GiB cache, and can drop QPS by 3–5×. Wrong pages cost more than a missed page. *(C3) One query cannot fill the device.* After host-side PQ navigation commits a rerank set, one query cannot rerank until that set's missing vector pages are materialized. The query exposes one finite Flash wave and then waits, leaving device parallelism unused even when bandwidth remains. That dependency is local to the query: bounded cross-query overlap can cover its wait with another query's navigation, materialization, or reranking, but cannot remove the single query's materialization dependency.

FlashANNS is a runtime for those three constraints, not a new graph index. A placement premise comes first: host DRAM holds graph adjacency and compact PQ codes, while full-precision vectors remain in the Flash-backed address space behind a bounded CXL-DRAM page cache. The first mechanism is a *wise prefetcher*. After PQ navigation commits the rerank candidates, it maps only their missing full-precision vectors to unique pages and issues those pages before exact scoring. An offline co-use-aware layout, online page deduplication, and density-bounded coalescing increase the fraction of each transferred page that reranking actually consumes; the prefetcher neither walks a speculative 2-hop neighborhood nor scores unused siblings to inflate utilization. The second mechanism is *continuous batching*. It overlaps one query's committed-page materialization with navigation or reranking from other queries, sustaining bounded Flash parallelism without changing per-query search semantics. Prefetch and batching may trigger Flash fills; the scoring thread may access the host cache or CXL-DRAM, but may not trigger or wait for Flash. We call that split *score hide*. The stronger contract—*query hide*, no wait for a fill on the timed path—is not a result we claim. Our evaluation therefore reports host-cache hits, CXL-DRAM hits, score-triggered Flash fills, and critical-path Flash wait before reporting iso-recall throughput. We do not use a full-corpus all-DRAM upper bound; absolute performance, matched-recall baselines, and the hide counters carry the evaluation.

This paper makes four contributions:

1. **A third serving point, scoped.** We place graph ANNS on a three-tier host-cache/CXL-DRAM-cache/Flash hierarchy and separate the *opportunity* (Flash-priced D) from the *system* (hide Flash fills from scoring). We do not claim the first CXL-DRAM ANNS or an evaluated Shared-FAM deployment (§2).
2. **Why naive CXL-SSD ANNS fails.** We measure the miss cliff, unwise page/prefetch admission, and single-query queue collapse, and we turn those negatives into design constraints (§3).
3. **A wise prefetcher plus continuous batching.** The prefetcher waits for candidate commitment, fetches only missing rerank-vector pages, and shapes them for high useful-page occupancy; batching overlaps those precise materialization

waves with independent work from other queries under bounded concurrency. Score hide requires zero score-triggered Flash fills and zero Flash wait during exact scoring (§4, §5).

4. **An iso-recall evaluation of hide, not just QPS.** We report the three-level hit/fill breakdown and critical-path Flash wait before iso-recall latency and throughput. Same-machine PipeANN is a named block-I/O comparison, not a hidden bar (§6).

2 Background

2.1 Graph ANNS walk

A graph ANNS index (Vamana [32], HNSW [25], or NSG [7]) stores, for each vector, a neighbor list of degree R. Search keeps a candidate set of width L (beam / `efSearch`). A *hop* expands the closest unread candidate u: the walk reads $N(u)$, scores those neighbors, and inserts the best unseen IDs back into the candidate set. The query returns the top-k after a hop or candidate budget is spent. Product quantization and its optimized variants compress the vectors used for approximate candidate ranking [8, 16]; FAISS and ScaNN provide widely used implementations of this quantized-search design space [14, 17].

One expand touches two kinds of bytes. Neighbor *IDs* are small and can live in host DRAM. Neighbor *vectors* are large: on Text2Image-10M each is 800 B, so scoring $N(u)$ moves about $R \times 800$ B if every neighbor is new. A typical hop is:

1. pick the closest unread candidate u from the beam;
2. read $N(u)$ (IDs only);
3. score the unseen neighbors' full-precision vectors;
4. insert the best new IDs into the candidate set.

Two facts follow for a memory hierarchy. First, the walk is *pointer-chasing*: step 3 of hop i decides which vector pages hop $i{+}1$ will need. Second, IDs need not live with vectors—the adjacency list can sit in host DRAM while embeddings sit on a slower tier. Recall depends on which useful neighbors the budget scores; stall rises with how many of those scores miss the fast tier. Widening L scores more nodes and cools the working set; that coupling is a measurement in §3, not a claim here.

2.2 The block-SSD serving stack

DiskANN and its maintained family [21, 31, 32] are the reference block-SSD design. Full-precision vectors and adjacency share 4 KB-aligned NVMe sectors. DRAM-resident PQ codes rank candidates without fetching full-precision vectors; an `aio` beam issues several sector reads at once; a static node cache retains entry and frequently visited vertices. PQ navigation exists to *avoid* full-precision reads, not to make NAND faster. Asynchronous I/O *hides* remaining reads by overlapping them with other work. Because graph and vectors share a sector, one 4 KB read is bound to whatever else the layout packed into that page.

Table 1. Latency tiers that a graph walk can see. CXL-DRAM is the published range; NAND fill is the order on the path we measure. The cliff itself is §3.

Access	Scale	Where
Host DRAM	$\sim 100\,ns$	graph / PQ metadata
Score-window hit	DRAM class	resident vectors
CXL-DRAM	140–410 ns	pooled graph / vectors
NAND fill	tens of μs	miss → flash

PipeANN [13] keeps that layout, overlaps compute with I/O inside one query, and can interleave queries to hold device queue depth. That cross-query interleaving is the block analogue of the continuous issue used later: its objects are I/O requests and sectors. The same idea will reappear on CXL-SSD as fetch-level issue (§4), against a faulting address space rather than `aio`.

2.3 CXL-DRAM ANNS

CXL.mem maps device HDM into the host physical address space [6]. Published Type-3 CXL-DRAM latencies sit in the 140–410 ns range [15, 33]. CXL-ANNS places the graph and embeddings in a CXL-DRAM pool and hides that far-memory delay with a relationship-aware cache and DSA-assisted prefetch. Cosmos [20] further offloads traversal and distance to device-side compute. Second-tier and residual designs [5, 28, 43] likewise assume a memory-class far tier without a per-miss NAND fill.

The interface is `load`/`store`; the capacity assumption is that D fits in CXL-DRAM. A service on that line is already usable in the sense that search never waits on flash. Prefetch and cache policies tuned for hundreds of nanoseconds do not transfer to a NAND fill of tens of microseconds. We keep the detailed contrast in §7.

2.4 Flash-backed CXL memory

In the device model studied here, a CXL-SSD pairs NAND capacity with a byte-addressable window [18, 22, 39, 41, 42]. Figure 1c is the deployment we serve. A *score-window hit* is served at DRAM-class latency. A *miss* is a NAND fill into that window, then a retry. The programming model looks like memory; the miss tax looks like flash. Table 1 places the three published tiers next to the NAND fill this paper must hide. The measured cliff belongs in §3; the table only fixes the orders of magnitude.

The window is bounded, and the admission unit is a page. On Text2Image-10M a vector is 800 B, so a 4 KB page holds several records—or several unused siblings. A blocking `load`/`store` miss and an explicit prefetch ioctl are both fills; they differ in *who waits*. If the scoring thread issues the fill, NAND is on the critical path. An early prefetch can overlap the fill; only a fill that completes before scoring leaves the scoring path. That split is the runtime contract in §4, not a result.

The prototype we measure is a software memory-semantic node (`vmem_sw`) in front of NVMe, with the score window implemented as host DRAM (`mbind`). It preserves fill semantics (a miss moves NAND bytes into the window), but not product CXL.mem timing. Hardware and generic OS work on this tier does not manage graph-ANNS residency; those systems belong in §7. Implementation limits belong in §5. Direct-attach CXL-SSD already gives one host flash-priced memory semantics; that is the deployment we measure. Multi-host one-copy under CXL 3.x Shared FAM is a premise we do not evaluate.

3 Motivation

Section 2 defined the walk and the fill. This section measures what happens if that fill is treated as a slightly slow DRAM load or as a DiskANN sector read. Each measurement asks two questions: does the effect exist, and what throughput or NAND-traffic penalty does it cause? The three results are the constraints Intro named C1–C3 (Table 2).

3.1 The miss cliff

A host `load` of one vector from CXL-DRAM is a few hundred nanoseconds. A matched-size CXL-SSD window-miss read is a NAND fill (Figure 2). On Text2Image-10M (800 B) the p_{50} is 0.28 μs versus 89 μs (∼320×). On a 10M-item LAION slice (2048 B) it is 0.67 μs versus 89 μs (∼130×). The miss is a 4 KB fill, so record size barely moves it; the CXL-DRAM hit scales with bytes.

A graph walk can pay that tax across hops and newly scored neighbors. Once a modest fraction of scores miss, wall-clock time is in the NAND path, not in distance arithmetic. DiskANN already knew the cliff for block I/O. The relevant distinction is that a memory-semantic interface does not remove it: `load`/`store` only changes *how* a miss is issued (§3.3). Treating CXL-SSD as memory-mapped DRAM therefore makes search miss-bound. The runtime contract that follows is not "make NAND faster"; it is that the scoring instruction must not wait on a fill.

3.2 Prefetch must be wise

Neighbor IDs are not sequential in the address space, so the next vector page is not the next byte. Coalescing

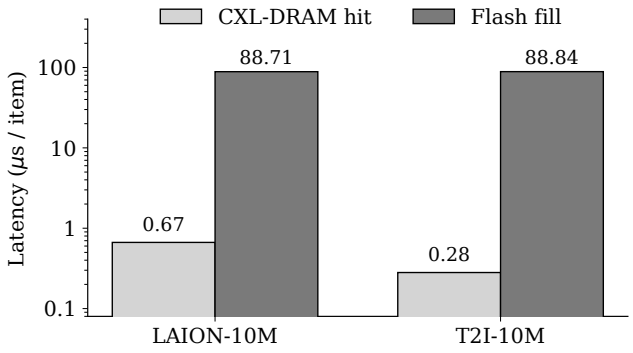


Figure 2. One-item host load: CXL-DRAM hit versus CXL-SSD cache miss. CXL-DRAM is Montage `/dev/dax0.0` (`clflush` then load). CXL-SSD is a `vmem_sw` window miss. T2I-10M uses the staged 800 B vectors. LAION-10M is the first 10M items of the 25M dump (d=512); the miss bar matches that item size on the same SSD path.

four 4 KB pages into one 16 KB admission unit therefore brings siblings the walk will not score. On a high-dimensional corpus where a record already fills most of a page, per-record admission versus page admission splits QPS by ∼3.6× and NAND reads by ∼3.7× (Figure 3a). At small dimensions several records share a page and the gap narrows; page-cache thinking is still the wrong default here.

Synchronous two-hop prefetch rooted at the top-1, top-2, or top-8 frontier candidates does not fix the cliff. Hit rate is almost unchanged, NAND reads multiply, and QPS drops by ∼4–5× at the widest setting (Figure 3b). Precision (*used*/*promoted*) falls as coverage grows; leftover pages evict useful ones from a small window. Wrong pages cost more than a missed page. A page can be necessary yet still waste most of its vector slots. The useful policy therefore delays full-precision movement until the approximate search commits its rerank set, then fetches only that payload and packs co-used vectors together—not "prefetch harder."

Widening L can become another form of unwise coverage. Under the fixed hop budget, recall and NAND reads both plateau from L=32 to 64. Extra width buys no measured quality benefit in that interval; quality and I/O cost must therefore be reported together.

3.3 One query cannot fill the device

Best-first search is dependency ordered even when its graph and PQ codes are host resident: approximate navigation must commit C_L before the runtime knows the exact full-precision payload to fetch, and reranking must wait until that payload is resident. A single query therefore exposes only one finite fill wave and then waits,

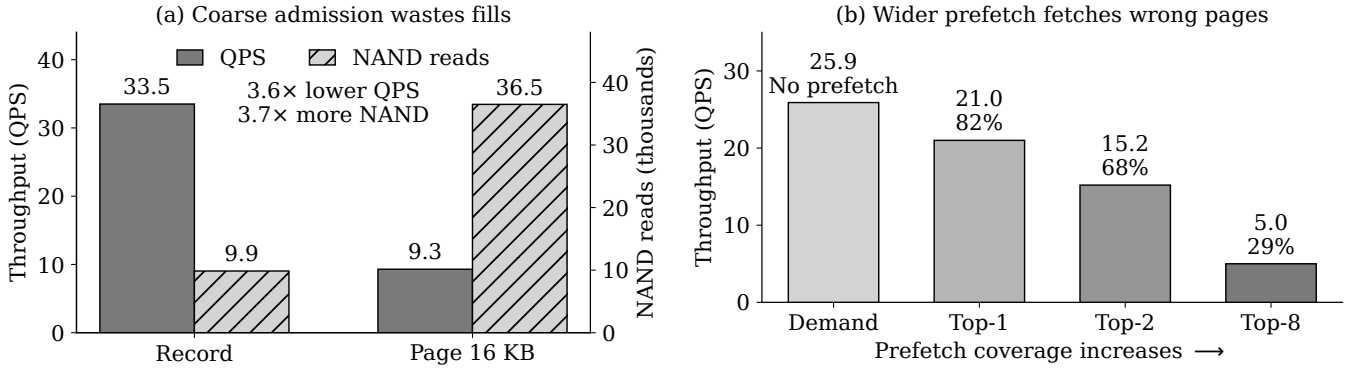


Figure 3. Unwise admission amplifies the cliff (LAION-200k, d=1024, hops=32). (a) A 16-KB admission unit generates 3.7× more NAND reads and cuts throughput by 3.6× relative to record admission. (b) Widening synchronous prefetch lowers promotion precision from 82% to 29% and throughput from 21.0 to 5.0 QPS; the unchanged 58% hit rate is omitted.

leaving device-queue bubbles before reranking (Figure 4). A block interface exposes I/O requests that a scheduler can interleave across independent queries. On CXL-SSD the programming model is a faulting address space. A blocking `load/store` (or a blocking `memcpy` from the mapping) is a synchronous NAND fault: effective queue depth collapses to ≈ 1 per thread. If a query waits for those fills before scoring, NAND latency remains on its critical path.

The idle device is a service-pipeline problem, not a bandwidth wall. The dependency is local to a query: while one query waits for its committed pages, another can navigate, materialize, or rerank. Bounded cross-query overlap can therefore sustain useful Flash work without changing which candidates any query visits or scores; the mechanism belongs in §4.

3.4 From measurements to constraints

Table 2 is the contract for §4. We do not treat CXL-SSD as uniform memory (C1): scoring must not touch NAND. Prefetch must be wise (C2): trigger at candidate commitment, fetch only the committed full-precision payload, and place co-used vectors so each page carries more data that reranking consumes. One query cannot fill the device (C3): bounded cross-query overlap must cover its materialization barrier with independent work from other queries.[1] Placement of neighbor IDs in host DRAM is a premise for C2, not a separate challenge: without it the prefetcher misses once just to learn whom to fetch.

[1]A small shared pin of entry vertices helps a narrow beam more than deepening one walk. That is an admission hint, not a positive LFU-hot-set module, and not a fourth challenge.

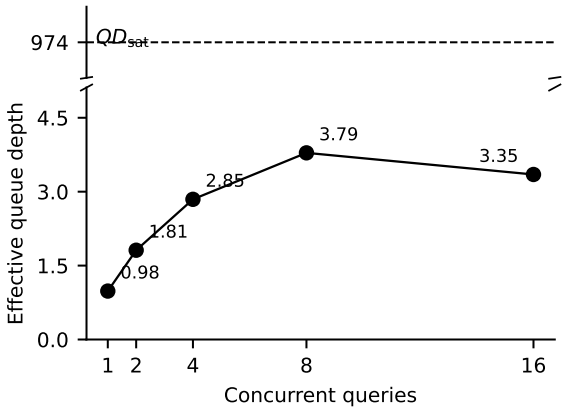


Figure 4. Per-query dependencies limit Flash I/O concurrency. Frozen PQ-64 end-batch on T2I-10M (nq=500, seed 42, iso-recall 0.920). $QD_{\text{eff}}(1) = 0.98$, $QD_{\text{eff}}(16) = 3.35$, measured $QD_{\text{sat}} = 974$ (4 KiB randread). One committed fill wave cannot hold the device; overlapping independent queries raise effective queue depth (peak 3.79 at $T = 8$). How the runtime issues across queries is §4.

4 Design

FlashANNS is a userspace ANNS runtime plus a thin residency plane in front of host DRAM, a bounded window that plays the role of CXL-DRAM, and a flash-backed CXL address space that holds D (Figure 5). Search is still best-first graph walk. The service contract is a *split*: prefetch and continuous batching may touch CXL-SSD; the scoring thread may not. A hop that computes a full-precision distance from bytes that were not already in the window is a hide failure, not a successful promote.

The wise prefetcher turns a committed PQ rerank set into one precise page batch before full-precision scoring.

Table 2. Measured pathologies become the three constraints Intro named. C1 uses the 10M-item probes in Figure 2; C2 uses LAION-200k; C3 follows from the dependency and issue path illustrated above.

ID	Exists + hurts	Constraint	FlashANNS answers
C1	Miss cliff ∼130–320×	Score only resident bytes	Score-hide contract
C2	Page ∼3.6×; blind prefetch ∼4–5×	Commit, select, and pack useful pages	Wise prefetcher
C3	One committed wave; blocking fill ⇒ idle Flash	Overlap independent queries	Continuous batching

That precision does not remove the query's materialization barrier: its rerank still waits for its own batch. Continuous batching exploits the independence across queries to cover that wait with other useful work. The modules below separate these two roles and the evaluation tests them independently.

4.1 D1: Semantic placement

Table 3 is the default map. The graph adjacency and the PQ codebook are touched on every hop and are small relative to the full-precision corpus; they live in host DRAM so they never compete with vectors for the CXL-SSD window (F2, F7). Query state (candidates, visited set, thread-local arenas) is likewise host-local. Full-precision vectors default to the CXL-SSD address space. A bounded *hub / shared hot set* of vector pages is explicitly cached into the window or into CXL-DRAM (§4.3). Only full-precision vectors in a committed rerank set become prefetch candidates; a discovered frontier does not (F3).

This is a deliberate difference from DiskANN, where adjacency and vectors share SSD sectors. We pay a modest host-DRAM tax for the graph in order to stop the window from turning into an accidental adjacency cache.

4.2 D2: Wise prefetch at candidate commitment

When: after commitment. The host first completes its approximate beam using the adjacency graph and PQ codes, without reading full-precision vectors from Flash. When that beam commits the bounded rerank set C_L, the prefetcher issues immediately: after uncertainty has been pruned, but before exact scoring dereferences any vector in C_L. This trigger deliberately gives up speculative lead time to avoid moving payload for candidates that will not survive. The single-query path may still wait for materialization to complete; the invariant is that the subsequent scoring loads do not themselves fault on Flash.

Table 3. Default placement. **Fill-in:** bytes on the corpus you report (e.g., 25M × dim).

Object	Default tier	Why
Graph (R neighbors)	host DRAM	every hop; keep off the window
PQ / codebook	host DRAM	tiny, extremely hot
Query / visited	host DRAM	per-worker state
Full-precision vec.	CXL-SSD	capacity
Entry / soft-pin	window (tiny)	F7; LFU hot set demoted
Committed rerank vec.	window via prefetch	F3, F5; score only if resident

What: the committed payload. The residency plane maps the IDs in C_L to their full-precision vector pages, removes pages that are resident or already in flight, and deduplicates the rest. Every requested page is therefore justified by at least one vector that exact reranking will consume. Adjacency, PQ codes, speculative neighbors, and unused frontier records are not part of this wave. Because Flash moves 4 KiB pages, however, a justified page may still carry vector slots that this query will not score.

How: make each page useful. Offline, a co-use-aware layout groups vectors that disjoint training searches tend to request together and places the resulting pages into short runs. Online, exact page filtering avoids duplicate movement, and the runtime fills gaps only in a short, sufficiently dense run; it never expands a sparse request into a whole SSD stripe. This combination targets useful payload per read rather than batch size alone. Unlike generic remote-memory or device-side sequential prefetchers [23, 26], the request is justified by the committed rerank set rather than an inferred address stream. We distinguish page precision (a fetched page contains any scored vector), vector-slot utilization (scored slots over all slots read), and extent utilization (requested pages over pages transferred after coalescing); vector-slot utilization is the direct measure of real in-page use.

4.3 D3: Entry pin and soft-pin (hot set demoted)

Single-query deepening is a bad way to spend a small window (F4, F7). A 64 MiB shared LFU set raised hit rate and *destroyed* QPS on a warm protocol (14.3→2.4); we do not ship it as D3. What remains is a tiny hard pin of the entry subgraph (≤32 MiB) and a hop-decaying

Fig. 6 placeholder — architecture:
top: Placement | Window admit | Prefetch+`cont` | L ctrl;
middle: residency plane (fill off crit path / score from window);
bottom: host DRAM (graph/PQ) | window = virtual CXL-DRAM | CXL-SSD.

Figure 5. FlashANNS architecture. From the host, scored bytes look like CXL-DRAM; capacity is CXL-SSD. The residency plane is the only path that moves full-precision bytes. Search never page-faults a neighbor "to be helpful." **Fill-in:** use shipped names (`PREFETCH_BATCH`, DramWindow); mark `vmem_sw`.

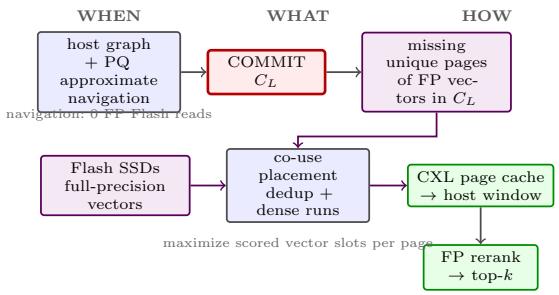


Figure 6. Wise prefetch answers three questions. *When*: after PQ navigation commits C_L. *What*: only missing unique pages of its full-precision vectors. *How*: co-use-aware placement, exact filtering, and dense short runs increase useful vector slots per Flash read.

soft-pin of recently installed pages. Pinning is never implicit in a page fault. Cross-query reuse, if it happens, is because committed materialization left useful pages in the window—not because an LFU daemon is competing with search.

4.4 D4: Hide engine — wise prefetch and continuous batching

A query-local barrier. D2 creates one precise materialization wave after PQ navigation commits C_L, but the same query cannot rerank until that wave completes. Better page selection alone therefore cannot keep Flash busy.

Cross-query overlap. The barrier is local to a query, not to the service: while one query waits, another can navigate, materialize its own committed wave, or rerank resident bytes. Continuous batching interleaves these stages so host work and Flash access coexist (Figure 7). Like continuous scheduling in other staged serving systems [40], it admits work at a semantic boundary—candidate commitment rather than a model iteration.

Bounded admission. The runtime bounds in-flight query stages and admits another committed wave as capacity becomes available, avoiding both device bubbles and an unbounded work queue. It neither predicts future candidates nor implements adaptive queue-depth control.

Semantic invariance. Each query retains the same graph, PQ codes, beam budget, and visited state, and reranks only after its own pages are resident. Batching changes timing, but not its candidate set or recall.

4.5 D5: Observability

Every experiment in §6 is required to report hide counters first: fraction of full-precision scores whose bytes were already in the window (`score_from_window`), critical-path NAND wait (`crit_wait_ns`), and device-fill/wall overlap, then QPS at the iso-recall floor. Prefetch efficiency is reported as page precision, vector-slot utilization, and extent utilization; a page touched once is not evidence that all of its payload was useful. QPS without those counters is not a hide result.

What we refuse to hide. If a module cannot move `crit_wait_ns` or `score_from_window` in an ablation, it is removed from the contribution list rather than kept as decoration. An LFU hot set that raises hit rate and drops QPS is already in that bin.

5 Implementation

FlashANNS is a C++17 userspace runtime that loads the search-ready adjacency graph and 64-byte PQ codes into host DRAM while keeping full-precision vectors in the Flash-backed `/dev/vmem0` address space. For each query, the host completes a PQ-ADC beam search, commits the bounded rerank set C_L, translates its vector IDs through the physical slot map, and removes duplicate and already-resident 4 KiB pages. The remaining pages are shaped into dense extent runs and submitted in chunks of at most 32 pages through a shared asynchronous copy pool; reads land in prefaulted bounce

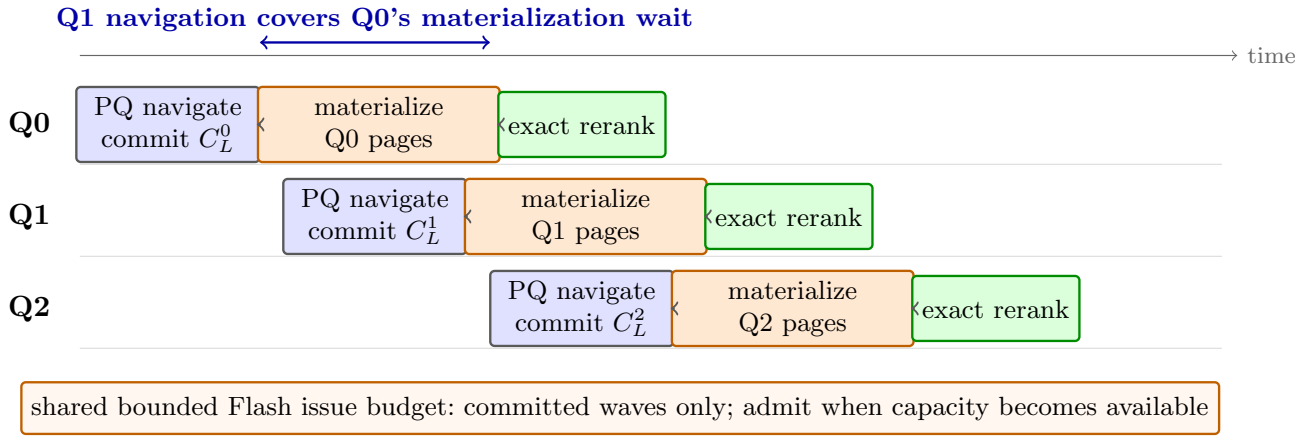


Figure 7. Continuous batching fills the service-level bubbles left by a single precise materialization wave. When Q0 waits for its committed pages, Q1 and Q2 contribute navigation, Flash materialization, or reranking under a bounded in-flight budget. Each query still crosses its own materialization barrier before reranking, so scheduling changes timing but not search semantics.

slabs before being installed in a bounded `DramWindow`. Exact MIPS or L2 reranking starts only after every page required by that query is resident, so scoring never dereferences an outstanding Flash page. This path implements the paper's three prefetch decisions directly: issue after candidate commitment, fetch only missing pages of committed vectors, and combine co-use placement, page deduplication, and bounded extent formation to raise useful bytes per transfer.

The default serving configuration uses eight CPU workers, a 128 MiB `DramWindow` and two query slots per worker, one shared copy pool, and a global issue depth of eight. Its completion-driven scheduler admits and issues work while depth is available, reranks any query whose committed pages are covered, and otherwise pumps completed transfers; only slot reservation is serialized, while entry selection and PQ-table construction remain query-local. Issue-depth accounting follows physical I/O waves rather than query lifetime, and the runtime rechecks and refills a committed page if it is evicted before reranking, preserving both termination and the candidate/recall contract. Our measured prototype realizes the CXL-side hierarchy with `vmem_sw`: a 3.87 TB memory-semantic mapping over two 1.92 TB Dell CD8P SSDs (2 MiB striping) and a 4 GiB software-managed page cache. The scoring windows reside in NUMA-bound host DRAM; therefore the reported results include proxy, bounce-copy, and cache-lock overhead and do not claim a physical CXL.mem SSD, FPGA-BAR fills, or device-side graph traversal.

6 Evaluation

We answer five questions:

- **Q1** Does naive memory-semantic access expose NAND stalls during graph search?
- **Q2** At matched recall, how do the original- and optimized-layout Demand baselines, PipeANN, and FlashANNS compare in mean latency and throughput?
- **Q3** Do the Wise Prefetcher and continuous batching each contribute to the result?
- **Q4** Does the benefit survive cold starts and changes in CXL-side cache capacity?
- **Q5** Where does FlashANNS still lose, and what resources does it consume?

6.1 Setup

We follow established ANNS evaluation practice [2, 30]: every performance comparison reports recall, and any speedup claim uses the same recall operating point. Table 4 summarizes the platform.

The primary configuration uses eight search threads. We sweep L to expose the recall–performance frontier rather than selecting whichever point makes a system look best. Error bars are 95% bootstrap intervals over five independent repetitions. Every internal run uses the same 4 GiB CXL-side cache and records host-window residency, CXL-side cache activity, NAND traffic, and critical-path wait. A run without a declared cold/warm state or five accepted repetitions is excluded.

Baselines. Both Demand baselines perform the same PQ-guided search, disable prefetching and continuous scheduling, and materialize missing full-precision pages synchronously. **Demand (original layout)** additionally removes the extent placement, while **Demand (optimized layout)** retains it. **PipeANN** is the same-machine block-I/O comparison with its own DiskANN/PipeANN

Table 4. Evaluation platform and fixed experiment protocol.

Item	Configuration
Host	Intel Xeon 6710E, 64 cores, 32 GiB DRAM; one NUMA node
Host-resident state	graph, PQ-64 codes, and 128 MiB scoring window per search thread
CXL-side cache	4 GiB `vmem_sw` page cache unless varied explicitly
Flash backing	two 1.92 TB Dell CD8P NVMe SSDs, 2 MiB striping, exposed through `/dev/vmem0`
Datasets	T2I-10M (200-d, MIPS), YFCC-10M (192-d, L2), LAION-10M (512-d, MIPS)
Index/search	10M vectors, Vamana R=32, PQ-64 navigation, committed full-precision reranking
Protocol	10,000 fixed queries; five repetitions per mark; cold runs begin at `cache_used` = 0

index and PQ navigation. **Wise-only** retains the Wise Prefetcher but disables cross-query continuous scheduling. We do not execute or plot an Oracle system.

6.2 Q1: Naive CXL-SSD fails

Figures 2–4 establish the failure mode used by the rest of the evaluation: a graph walk that synchronously crosses the memory–Flash boundary repeatedly exposes NAND latency and fails to build device concurrency. Q2 therefore measures complete search, while Q3 and Q4 separate the decisions that remove this wait from the scoring path.

6.3 Q2: End-to-end iso-recall

Figure 8 reports both required views of the end-to-end frontier. Both Demand variants and FlashANNS have identical recall at each L because they preserve the same navigation and candidate set; they differ only in placement, materialization, and scheduling. PipeANN is plotted at its measured operating points instead of being interpolated into an artificial match.

At these matched-recall points, throughput improves over Demand (optimized layout) by 1.83× on LAION, 2.34× on T2I, and 2.45× on YFCC. The optimized layout alone improves Demand throughput over its original layout by 3.5%, 8.1%, and 5.6%, respectively; the remaining gain therefore cannot be attributed to placement. The figures also expose an important boundary: PipeANN remains faster at several operating points, particularly on LAION-10M. We treat it as a block I/O performance reference, not as evidence for the deployment property of the memory interface.

6.4 Q3: Each module is necessary

Figure 9 isolates the Wise Prefetcher at one thread so cross-query scheduling cannot explain its gain. Issuing after candidate commitment removes most blocking time relative to serialized materialization; fetching only missing committed pages bounds the host-window requests; and forming extents reduces physical NAND commands without changing the candidate IDs or recall. Figure 10 separately quantifies the layout and physical-I/O effects with matched-concurrency controls.

Figure 9 then restores concurrency. It compares the same Wise Prefetcher with and without continuous batching from T=1 through T=16 on every dataset. The dotted line and filled points identify the primary T=8 configuration. At T=8, full FlashANNS reaches 600.8 QPS on T2I, 3016.2 QPS on YFCC, and 421.2 QPS on LAION. Wise-only reaches 559.2, 2933.2, and 406.0 QPS, respectively. Thus the batching contribution is measurable but smaller than the prefetcher's removal of synchronous NAND wait; Figure 11 tests the separate service-level question under open-loop arrivals.

Co-use placement reduces host-window requests by 5.2% and raises useful vector-slot occupancy from 61.46% to 64.16% at the same T=8 concurrency. Across the separately controlled T=1 prefetcher stages, post-commit batching reduces physical bytes and read amplification by 2.5% relative to serialized materialization; extent formation then reduces NAND commands by a further 7.0% while increasing the mean transfer from 4.05 to 4.35 KiB. End to end, the selected path uses 8.3% fewer physical NAND bytes and 15.7% fewer NAND commands than the original-layout control.

On T2I, FlashANNS sustains 368.9 of 369.2 offered QPS at the 1.25× point with 77.7 ms P99, whereas PipeANN saturates at 295.0 QPS and its P99 rises to 6.75 s. The other datasets expose the boundary rather than a universal win: FlashANNS tracks the reference rate on YFCC (2930.5 of 2933.2 QPS) and nearly tracks it on LAION (395.5 of 406.0 QPS), but both saturate at 1.25× and PipeANN retains substantially lower tail latency. Continuous batching reduces P99 versus Wise-only at the reference rate on T2I and YFCC; LAION's larger payload reaches its I/O/queueing limit earlier.

6.5 Q4: Cold hide and cache sensitivity

Figure 12(a) is the direct hide test. Demand exposes 20.40, 3.54, and 19.74 ms of critical NAND wait per query on T2I, YFCC, and LAION; the corresponding FlashANNS counter is zero because scoring consumes the host window only after its asynchronous fill completes. Cold and immediate warm results remain close, so the result is not created by an undeclared warmup. Increasing the T2I CXL-side cache from 1 to 8 GiB raises

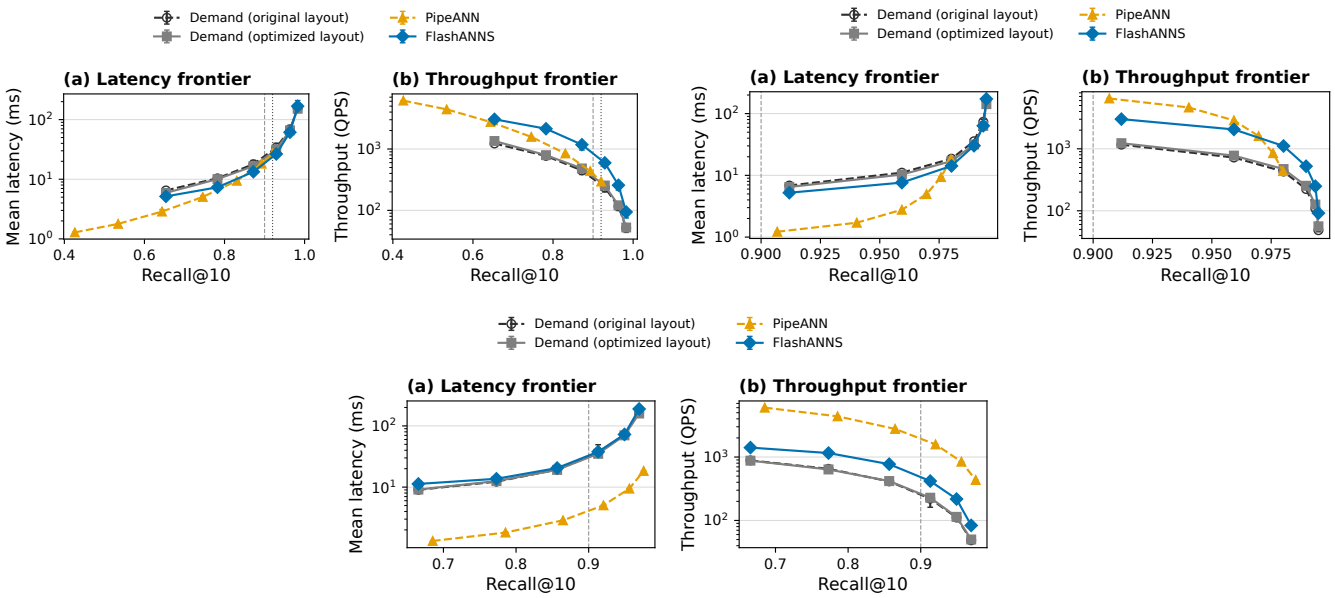


Figure 8. Recall–latency and recall–throughput frontiers on T2I-10M (top left), YFCC-10M (top right), and LAION-10M (bottom). Each mark summarizes five true-cold runs of 10,000 queries; error bars are 95% bootstrap intervals. At the representative matched-recall points, FlashANNS sustains 595.2/3013.2/418.0 QPS, versus 254.4/1232.2/228.2 QPS for Demand (optimized layout). The original-layout Demand curve isolates the benefit of extent placement.

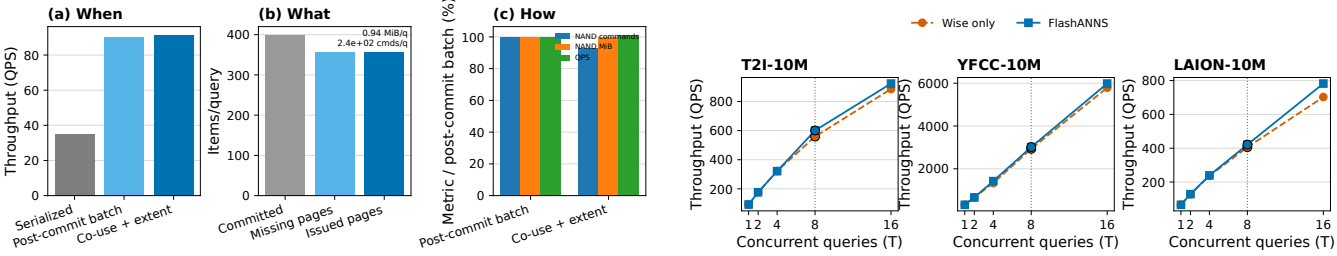


Figure 9. Causal evaluation of the two designs. Left: Wise Prefetcher ablation on T2I-10M at L=400 and T=1, answering when, what, and how to prefetch. Right: continuous-batching scaling for $T \in \{1, 2, 4, 8, 16\}$ on all three datasets; the vertical guide marks the primary T=8 configuration. Every mark summarizes five cold runs.

throughput from 466.1 to 666.1 QPS, while the fixed 4 GiB point delivers 595.5 QPS.

6.6 Q5: Losses, overhead, and fairness

FlashANNS does not dominate a tuned block-I/O engine at every recall point. PipeANN is faster where its compact traversal and block interface are a better fit, while FlashANNS pays for the host graph, PQ codes, per-thread scoring windows, CXL-side cache management, and memory-semantic software path. The benefit also narrows when the working set is already hot. These costs delimit the claim: FlashANNS shows how to hide Flash fills behind graph-ANNS work in the three-tier CXL organization; it does not claim that memory semantics erase all software overhead or make NAND equivalent to DRAM.

Takeaway. Across three 10M-vector datasets, the Wise Prefetcher removes synchronous NAND wait from committed scoring, while continuous batching supplies a smaller, separately measured scaling benefit through the primary eight-thread serving configuration.

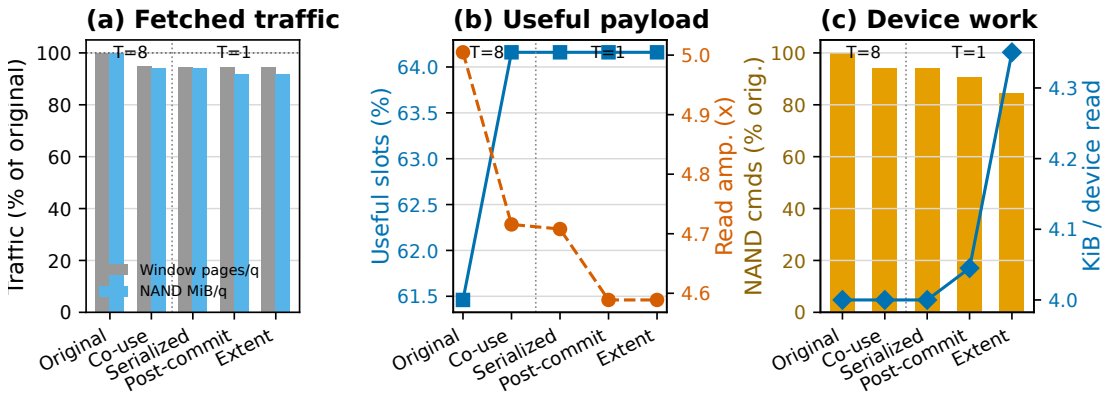


Figure 10. Wise Prefetcher I/O efficiency on T2I-10M at L=400. The dotted separator distinguishes the T=8 layout control (Original versus Co-use) from the T=1 prefetcher control (Serialized, Post-commit, and Extent). Window pages are host-window requests, not NAND misses; NAND bytes and commands come from physical device counters. Read amplification is physical NAND bytes divided by the useful exact-vector payload. Every mark summarizes five accepted 10,000-query runs with identical query and candidate hashes.

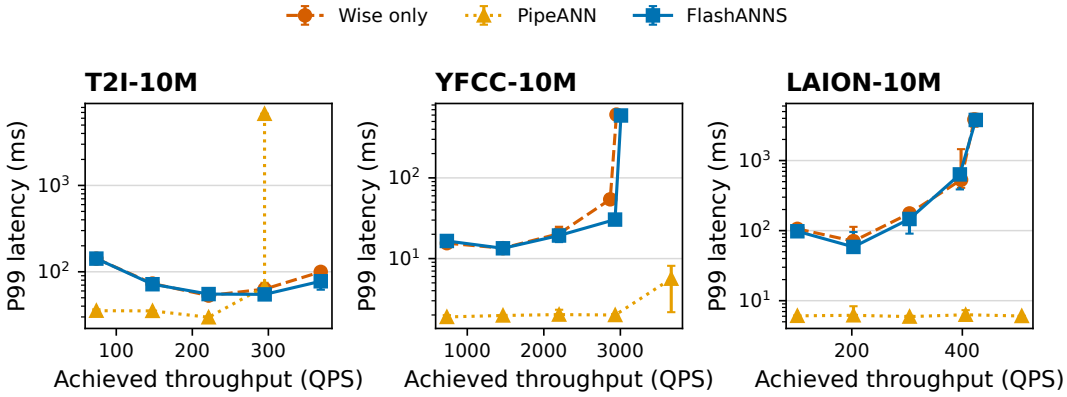


Figure 11. Open-loop service curves at T=8. Each panel sweeps five frozen offered rates from 0.25× to 1.25× the measured reference saturation rate and plots achieved throughput against end-to-end P99 latency, including queueing. Every mark is the median of five accepted 10,000-query runs.

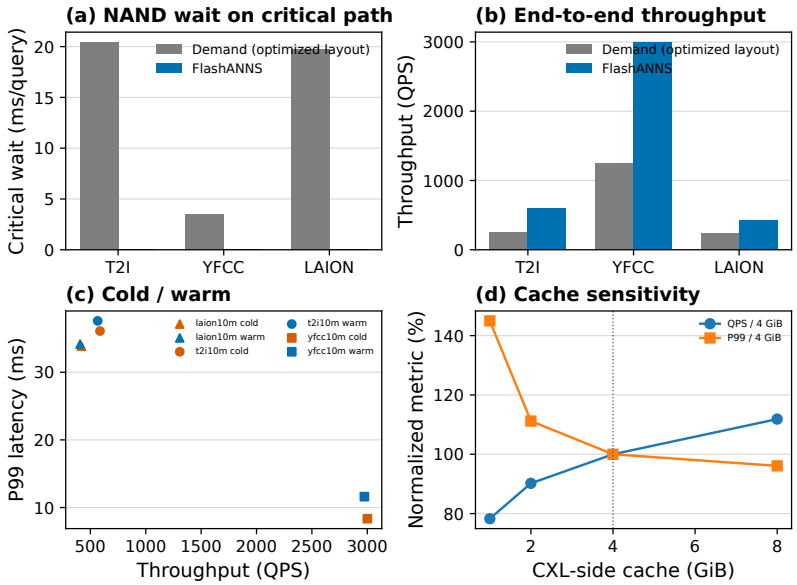


Figure 12. Critical-path hiding and robustness. (a–b) Demand versus FlashANNS at the representative matched-recall point on each dataset. (c) A true-cold run and its immediate warm partner. (d) T2I-10M sensitivity to the CXL-side cache, normalized to the fixed 4 GiB configuration. Every mark summarizes five runs.

7 Related Work

Table 5 is the comparison we want a reviewer to use. Prose below only records the one-line difference; it does not repeat the background.

Block-SSD ANNS. DiskANN, FreshDiskANN, and PipeANN optimize PQ navigation and I/O overlap on block devices [13, 31, 32]; SPANN and Starling explore hybrid or disk-resident layouts [4, 36]. SPFresh, Filtered-DiskANN, and Milvus extend this space to updates, filters, and serving [9, 35, 38]. Unlike an I/O scheduler, FlashANNS manages a memory-semantic window and the committed vector pages required before exact reranking.

Table 5. Closest systems and their defining distinction.

System	Slow tier	Primary distinction
DiskANN / PipeANN	block SSD	PQ navigation + async block I/O
Second-Tier / FaTRQ	far memory	coarse/fine far reads
CXL-ANNS / Cosmos	CXL DRAM / PNM	memory latency or compute offload
CXL-SSD hardware	NAND + window	generic memory-semantic access
FlashANNS	**CXL-SSD**	**committed pages + cross-query overlap**

CXL-DRAM and near-memory ANNS. CXL-ANNS, Cosmos, and second-tier/residual designs assume a DRAM- or NVM-priced far tier [5, 15, 20, 28, 43]; CXL-aware vector-database restructuring makes the same memory-class assumption [19]. Their ∼100 ns policies do not address tens-of-microseconds NAND behind a polluted window (F3), and FlashANNS does not require near-memory compute.

Far-memory runtimes and CXL tiering. Remote paging and application-integrated runtimes manage far capacity through paging, isolation, and prefetch [3, 12, 26, 29, 34]; CXL tiering places hot pages across DRAM-class tiers [1, 24, 27, 37, 44, 45]. Those policies react to generic locality; FlashANNS uses PQ commitment to identify reranking's full-precision payload.

CXL-SSD hardware and generic software. DirectCXL and memory pooling establish CXL expansion [10, 11]; CXL-enabled SSD, SkyByte, From-Block-to-Byte, and CMM-H study flash-backed memory [22, 39, 41, 42]. They do not jointly manage graph-ANNS residency, committed-vector materialization, and cross-query overlap.

8 Discussion and Limitations

Proxy versus product. The measured path is FPGA `nvmex` plus `vmem_sw`, not a shipping CXL.mem SSD; the FPGA BAR is not the evaluated window. Shared FAM and one-copy provisioning are likewise unmeasured; device IOPS would remain shared.

Hide is the service contract, not a fallback. A usable service means exact scoring reads resident bytes (`score_from_window` = 100%) and `crit_wait_ns` is DRAM-class. Graph/PQ live in host DRAM; after candidate commitment, wise prefetch materializes the rerank pages and continuous issue schedules those batches across queries. A window miss while scoring is a hide failure we close, not a reason to shrink the claim.

Block-path comparison. PQ navigation plus deep `aio` avoids most full-precision reads. FlashANNS also uses PQ navigation, but materializes a committed FP

rerank set through a load/store hierarchy rather than issuing traversal records through block I/O. We therefore compare matched-recall frontiers and resource costs rather than normalizing against an unmatched all-DRAM run. Remaining costs include Flash materialization and proxy copying under a cache lock; scaling beyond the fixed budget and removing that copy are engineering questions, not scheduling claims.

What would change our mind. If HPS makes the FPGA BAR a coherent miss window, we re-measure hide there and keep the same counters. If hide counters never leave NAND-class wait, contribution 3 shrinks to "overlap vs. demand" and the intro must not say the host is unaware of CXL-SSD.

9 Conclusion

Flash-backed CXL memory offers flash-priced capacity through a memory-semantic window, but is unusable if treated as slow DRAM or mmap'd DiskANN. FlashANNS separates search information from payload movement: wise prefetch materializes the missing pages of a committed rerank set, and continuous batching overlaps each query's materialization with independent work. Thus exact scoring reads resident bytes while the remaining query-local barrier stays explicit.

References

[1] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can Far Memory Improve Job Throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3342195.3387522

[2] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2017. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. In *Similarity Search and Applications*. Springer, Cham, Switzerland, 34–49. doi:10.1007/978-3-319-68474-1_3

[3] Irina Calciu, M. Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. 2021. Rethinking Software Runtimes for Disaggregated Memory. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 79–92. doi:10.1145/3445814.3446713

[4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA, 5199–5212. https://proceedings.neurips.cc/paper/2021/hash/2cf49e22106c288c95c9d94bb9b0739a-Abstract.html

[5] Rongxin Cheng, Yifan Peng, Xingda Wei, Hongrui Xie, Rong Chen, Sijie Shen, and Haibo Chen. 2024. Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search and Breaking It with Second-Tier Memory. arXiv:2405.03267 doi:10.48550/arXiv.2405.03267

[6] Compute Express Link Consortium 2023. *Compute Express Link Specification, Revision 3.1*. Compute Express Link Consortium. https://www.computeexpresslink.org/spec-landing-page-compute-express-link

[7] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754

[8] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Los Alamitos, CA, USA, 2946–2953. doi:10.1109/CVPR.2013.379

[9] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, 3406–3416. doi:10.1145/3543507.3583552

[10] Donghyun Gouk, Miryeong Kwon, Hanyeoreum Bae, Sangwon Lee, and Myoungsoo Jung. 2023. Memory Pooling With CXL. *IEEE Micro* 43, 2 (2023), 48–57. doi:10.1109/MM.2023.3237491

[11] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct Access, High-Performance Memory Disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference*. USENIX Association, USA, 287–294. https://www.usenix.org/conference/atc22/presentation/gouk

[12] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. 2017. Efficient Memory Disaggregation with Infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, USA, 649–667. https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/gu

[13] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *19th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 171–186. https://www.usenix.org/conference/osdi25/presentation/guo

[14] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Online, 3887–3896. https://proceedings.mlr.press/v119/guo20h.html

[15] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference*. USENIX Association, USA, 585–600. https://www.usenix.org/conference/atc23/presentation/jang

[16] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on*

Big Data 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572

[18] Myoungsoo Jung. 2022. Hello Bytes, Bye Blocks: PCIe Storage Meets Compute Express Link for Memory Expansion (CXL-SSD). In *14th ACM Workshop on Hot Topics in Storage and File Systems*. Association for Computing Machinery, New York, NY, USA, 45–51. doi:10.1145/3538643.3539745

[19] Kyungbin Kim, Sungsu Ahn, Wonjung Jeong, Jongmin Kim, Sangun Choi, Minseong Gil, Minseong Kim, Dongha Jung, Yunjay Hong, Haekang Jung, and Yunho Oh. 2026. Bauhaus: Restructuring Vector Database for LLM Retrieval on CXL-Based Tiered Memory. *IEEE Trans. Comput.* 75, 4 (2026), 1309–1322. doi:10.1109/TC.2026.3656215

[20] Seoyoung Ko, Hyunjeong Shim, Wanju Doh, Sungmin Yun, Jinin So, Yongsuk Kwon, Sang-Soo Park, Si-Dong Roh, Minyong Yoon, Taeksang Song, and Jung Ho Ahn. 2025. Cosmos: A CXL-Based Full In-Memory System for Approximate Nearest Neighbor Search. *IEEE Computer Architecture Letters* 24, 1 (2025), 173–176. doi:10.1109/LCA.2025.3570235

[21] Ravishankar Krishnaswamy, Magdalen Dobson Manohar, and Harsha Vardhan Simhadri. 2024. The DiskANN Library: Graph-Based Indices for Fast, Fresh and Filtered Vector Search. *IEEE Data Engineering Bulletin* 48, 3 (2024), 20–42. http://sites.computer.org/debull/A24sept/p20.pdf

[22] Miryeong Kwon, Donghyun Gouk, Junhyeok Jang, Jinwoo Baek, Hyunwoo You, Sangyoon Ji, Hongjoo Jung, Junseok Moon, Seungkwan Kang, Seungjun Lee, and Myoungsoo Jung. 2025. From Block to Byte: Transforming PCIe Solid-State Devices With Compute Express Link Memory Protocol and Instruction Annotation. *IEEE Micro* 45, 6 (2025), 46–55. doi:10.1109/MM.2025.3581448

[23] Miryeong Kwon, Sangwon Lee, and Myoungsoo Jung. 2023. Cache in Hand: Expander-Driven CXL Prefetcher for Next Generation CXL-SSD. In *15th ACM Workshop on Hot Topics in Storage and File Systems*. Association for Computing Machinery, New York, NY, USA, 24–30. doi:10.1145/3599691.3603406

[24] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, New York, NY, USA, 574–587. doi:10.1145/3575693.3578835

[25] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473

[26] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively Prefetching Remote Memory with Leap. In *2020 USENIX Annual Technical Conference*. USENIX Association, USA, 843–857. https://www.usenix.org/conference/atc20/presentation/al-maruf

[27] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit O. Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. Association for Computing Machinery, New York, NY, USA, 742–755. doi:10.1145/3582016.3582063

[28] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 10672–10684. https://proceedings.neurips.cc/paper/2020/hash/788d986905533aba051261497ecffcbb-Abstract.html

[29] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K. Aguilera, and Adam Belay. 2020. AIFM: High-Performance, Application-Integrated Far Memory. In *14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 315–332. https://www.usenix.org/conference/osdi20/presentation/ruan

[30] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, Online, 177–189. https://proceedings.mlr.press/v176/simhadri22a.html

[31] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613 doi:10.48550/arXiv.2105.09613

[32] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 13748–13758. https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html

[33] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *56th IEEE/ACM International Symposium on Microarchitecture*. Association for Computing Machinery, New York, NY, USA, 105–121. doi:10.1145/3613424.3614256

[34] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Yiying Zhang, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2023. Canvas: Isolated and Adaptive Swapping for Multi-Applications on Remote Memory. In *20th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, USA, 161–179. https://www.usenix.org/conference/nsdi23/presentation/wang-chenxi

[35] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550

[36] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu,

Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27. doi:10.1145/3639269

[37] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. Nomad: Non-Exclusive Memory Tiering via Transactional Page Migration. In *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 19–35. https://www.usenix.org/conference/osdi24/presentation/xiang

[38] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 545–561. doi:10.1145/3600006.3613166

[39] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhyung Park, Jin yong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S. Kim. 2023. Overcoming the Memory Wall with CXL-Enabled SSDs. In *2023 USENIX Annual Technical Conference*. USENIX Association, USA, 601–617. https://www.usenix.org/conference/atc23/presentation/yang-shao-peng

[40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[41] Jianping Zeng, Shuyi Pei, Da Zhang, Yuchen Zhou, Amir Beygi, Xuebin Yao, Ramdas Kachare, Tong Zhang, Zongwang Li, Marie Nguyen, Rekha Pitchumani, Yang Soek Ki, and Changhee Jung. 2025. Performance Characterizations and Usage Guidelines of Samsung CMM-H. *IEEE Micro* 45, 5 (2025), 94–102. doi:10.1109/MM.2025.3591577

[42] Haoyang Zhang, Yuqi Xue, Yirui Eric Zhou, Shaobo Li, and Jian Huang. 2025. SkyByte: Architecting an Efficient Memory-Semantic CXL-based SSD with OS and Hardware Co-design. In *2025 IEEE International Symposium on High Performance Computer Architecture*. IEEE, Los Alamitos, CA, USA, 577–593. doi:10.1109/HPCA61900.2025.00051

[43] Tianqi Zhang, Flavio Ponzina, and Tajana Rosing. 2026. FaTRQ: Tiered Residual Quantization for LLM Vector Search in Far-Memory-Aware ANNS Systems. arXiv:2601.09985 doi:10.48550/arXiv.2601.09985

[44] Yuhong Zhong, Daniel S. Berger, Carl A. Waldspurger, Ryan Wee, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D. Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 37–56. https://www.usenix.org/conference/osdi24/presentation/zhong-yuhong

[45] Zhe Zhou, Yiqi Chen, Tao Zhang, Yang Wang, Ran Shu, Shuotao Xu, Peng Cheng, Lei Qu, Yongqiang Xiong, Jie Zhang, and Guangyu Sun. 2024. NeoMem: Hardware/Software Co-Design for CXL-Native Memory Tiering. In *57th IEEE/ACM International Symposium on Microarchitecture*. IEEE, Los Alamitos, CA, USA, 1518–1531. doi:10.1109/MICRO61859.2024.00109